

Programmation C

6 – Gestion dynamique de la mémoire

Alain CROUZIL, Emmanuel LAVINAL

`alain.crouzil@irit.fr, emmanuel.lavinal@irit.fr`

Département d'Informatique
Faculté Sciences et Ingénierie
Université de Toulouse

Licence
Semestre pair 2025-2026

 | Département
Informatique

 Faculté
sciences et
ingénierie
Université
de Toulouse

Sommaire

- 1 Introduction
- 2 Fonctions de gestion dynamique de la mémoire
- 3 Accès à la taille d'un objet
- 4 Durée d'allocation des emplacements en mémoire
- 5 Exercices

Sommaire

- 1 Introduction
- 2 Fonctions de gestion dynamique de la mémoire
- 3 Accès à la taille d'un objet
- 4 Durée d'allocation des emplacements en mémoire
- 5 Exercices

Introduction

Objectifs

La gestion dynamique de la mémoire permet :

- d'utiliser des objets dont la taille n'est connue qu'au moment de l'exécution ;
- de manipuler des emplacements en mémoire alloués et libérés selon les besoins.

Remarques

- L'accès aux emplacements en mémoire alloués de manière dynamique se fait grâce aux pointeurs.
- Ces emplacements sont généralement situés dans une zone de la mémoire appelée le tas (*heap*).

Sommaire

- 1 Introduction
- 2 Fonctions de gestion dynamique de la mémoire**
- 3 Accès à la taille d'un objet
- 4 Durée d'allocation des emplacements en mémoire
- 5 Exercices

Fonctions

#include <stdlib.h>

- **void** *malloc(size_t *taille*) alloue *taille* octets consécutifs et retourne l'adresse de l'emplacement, c'est-à-dire l'adresse du premier octet, ou NULL s'il y a un problème.
- **void** *calloc(size_t *nb_blocs*, size_t *taille*) alloue *nb_blocs* blocs consécutifs de *taille* octets, les initialise à zéro binaire (tous les bits à 0) et retourne l'adresse de l'emplacement ou NULL s'il y a un problème.
- **void** *realloc(**void** **p*, size_t *taille*) modifie la taille de l'emplacement d'adresse *p* (déjà alloué dynamiquement), *taille* étant la nouvelle taille, et retourne l'adresse du nouvel emplacement (différente ou pas de l'ancienne) ou NULL s'il y a un problème (l'ancien emplacement n'est alors pas libéré).

Remarques : realloc(NULL,n) $\Leftrightarrow$ malloc(n)

 realloc(p,0) $\Leftrightarrow$ free(p)

 realloc(p,0) retourne NULL

- **void** free(**void** **p*) libère l'emplacement d'adresse *p* (doit avoir été obtenue préalablement par un appel à malloc, calloc ou realloc).

Sommaire

- 1 Introduction
- 2 Fonctions de gestion dynamique de la mémoire
- 3 Accès à la taille d'un objet**
- 4 Durée d'allocation des emplacements en mémoire
- 5 Exercices

Accès à la taille d'un objet

Opérateur **sizeof**

La valeur, de type `size_t`, donnée par l'opérateur **sizeof** est la taille en octets de la zone en mémoire occupée par :

- une variable d'un type donné : **sizeof** *type*
Exemple : **sizeof**(`int *`)
- la valeur d'une expression donnée (à utiliser avec précaution) :
sizeof *expression*

Exemples d'utilisation

- Allocation d'un tableau de **int** :
`int n; scanf("%d",&n);`
`int *tab;`
`tab=malloc(n*sizeof(int)); // ou tab=calloc(n,sizeof(int));`
`tab[0]=12;`
- Allocation d'une structure :
`struct sPoint {double x,y; struct sPoint *pSuivant};`
`struct sPoint *p;`
`p=malloc(sizeof(struct sPoint));`
`p->pSuivant=NULL;`

Sommaire

- 1 Introduction
- 2 Fonctions de gestion dynamique de la mémoire
- 3 Accès à la taille d'un objet
- 4 Durée d'allocation des emplacements en mémoire**
- 5 Exercices

Durée d'allocation des emplacements en mémoire (1/2)

Principe

Une zone mémoire allouée dynamiquement le reste jusqu'à :

- ce qu'elle soit libérée par l'appel à la fonction `free` ;
- ce qu'elle soit libérée par un appel à la fonction `realloc` aboutissant à un déplacement de la zone mémoire et donc à l'allocation d'une nouvelle zone ;
- la fin de l'exécution du programme (à éviter).

Durée d'allocation des emplacements en mémoire (2/2)

Exemple

```
void f(void)
```

```
{  
    int *p;  
    p=malloc(sizeof(int));  
    *p=12;  
}
```

```
int main(void) ←
```

```
{  
    f();  
    // Ici, p est inconnue,  
    // l'entier alloué existe toujours en mémoire,  
    // mais il n'est plus accessible (perdu).  
    printf("Terminé\n");  
  
    return 0;  
}
```

Durée d'allocation des emplacements en mémoire (2/2)

Exemple

```
void f(void)
```

```
{  
    int *p;  
    p=malloc(sizeof(int));  
    *p=12;  
}
```

```
int main(void)
```

```
{  
    f();  
    // Ici, p est inconnue,  
    // l'entier alloué existe toujours en mémoire,  
    // mais il n'est plus accessible (perdu).  
    printf("Terminé\n");  
  
    return 0;  
}
```

Durée d'allocation des emplacements en mémoire (2/2)

Exemple

```
void f(void) ←
```

```
{  
    int *p;  
    p=malloc(sizeof(int));  
    *p=12;  
}
```

```
int main(void)
```

```
{  
    f();  
    // Ici, p est inconnue,  
    // l'entier alloué existe toujours en mémoire,  
    // mais il n'est plus accessible (perdu).  
    printf("Terminé\n");  
  
    return 0;  
}
```

Durée d'allocation des emplacements en mémoire (2/2)

Exemple

```
void f(void)
```

```
{  
  int *p; ←  
  p=malloc(sizeof(int));  
  *p=12;  
}
```

```
int main(void)
```

```
{  
  f();  
  // Ici, p est inconnue,  
  // l'entier alloué existe toujours en mémoire,  
  // mais il n'est plus accessible (perdu).  
  printf("Terminé\n");  
  
  return 0;  
}
```



Fonction appelée

Fonction appelante

Durée d'allocation des emplacements en mémoire (2/2)

Exemple

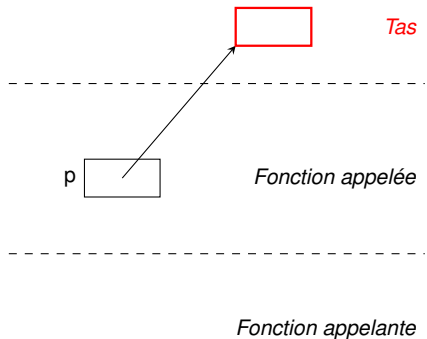
```
void f(void)
```

```
{
  int *p;
  p=malloc(sizeof(int));
  *p=12;
}
```

```
int main(void)
```

```
{
  f();
  // Ici, p est inconnue,
  // l'entier alloué existe toujours en mémoire,
  // mais il n'est plus accessible (perdu).
  printf("Terminé\n");

  return 0;
}
```



Durée d'allocation des emplacements en mémoire (2/2)

Exemple

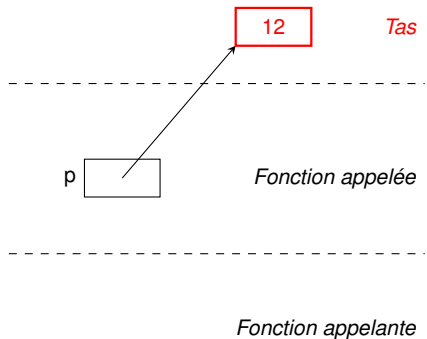
```
void f(void)
```

```
{  
    int *p;  
    p=malloc(sizeof(int));  
    *p=12;   
}
```

```
int main(void)
```

```
{  
    f();  
    // Ici, p est inconnue,  
    // l'entier alloué existe toujours en mémoire,  
    // mais il n'est plus accessible (perdu).  
    printf("Terminé\n");
```

```
    return 0;  
}
```



Durée d'allocation des emplacements en mémoire (2/2)

Exemple

```
void f(void)
```

```
{  
    int *p;  
    p=malloc(sizeof(int));  
    *p=12;  
} ←
```

12

Tas

```
int main(void)
```

```
{  
    f();  
    // Ici, p est inconnue,  
    // l'entier alloué existe toujours en mémoire,  
    // mais il n'est plus accessible (perdu).  
    printf("Terminé\n");
```

Fonction appelée

```
    return 0;
```

Fonction appelante

Durée d'allocation des emplacements en mémoire (2/2)

Exemple

```
void f(void)
```

```
{
  int *p;
  p=malloc(sizeof(int));
  *p=12;
}
```

12

Tas

```
int main(void)
```

```
{
  f();
  // Ici, p est inconnue,
  // l'entier alloué existe toujours en mémoire,
  // mais il n'est plus accessible (perdu).
  printf("Terminé\n");
```

Fonction appelée

```
return 0;
```

Fonction appelante

Affichage de Terminé

Durée d'allocation des emplacements en mémoire (2/2)

Exemple

```
void f(void)
```

```
{  
    int *p;  
    p=malloc(sizeof(int));  
    *p=12;  
}
```

```
int main(void)
```

```
{  
    f();  
    // Ici, p est inconnue,  
    // l'entier alloué existe toujours en mémoire,  
    // mais il n'est plus accessible (perdu).  
    printf("Terminé\n");  
  
    return 0;   
}
```

Sommaire

- 1 Introduction
- 2 Fonctions de gestion dynamique de la mémoire
- 3 Accès à la taille d'un objet
- 4 Durée d'allocation des emplacements en mémoire
- 5 Exercices**

Exercices (1/3)

Exercice 6.1 Allocation dynamique d'un tableau

Écrivez un programme qui stocke des valeurs tapées par l'utilisateur dans un tableau d'éléments de type **double** alloué de manière dynamique, puis affiche ces valeurs à l'écran. Pour cela, le programme doit :

- demander à l'utilisateur le nombre de valeurs qu'il va saisir ;
- allouer le tableau ;
- saisir les valeurs en les stockant dans le tableau ;
- afficher les valeurs stockées dans le tableau ;
- libérer l'espace mémoire occupé par le tableau.

Exercices (2/3)

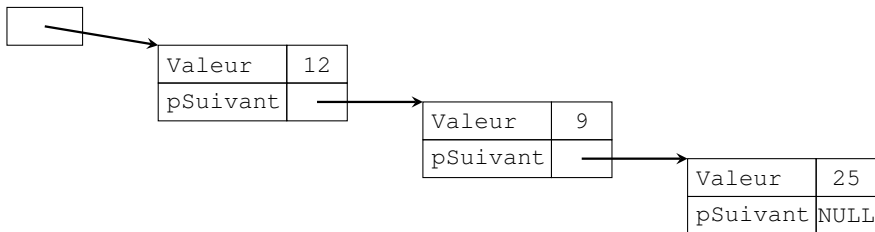
 Exercice 6.2 Liste chaînée

On souhaite manipuler des listes d'entiers sous la forme la plus basique de liste chaînée dynamique. Pour cela :

- une liste est représentée par un pointeur vers la première cellule de la liste ;
- une liste vide est représentée par la valeur NULL de ce pointeur ;
- chaque cellule est une structure contenant l'entier de type `int` ainsi que l'adresse de la cellule suivante ;
- pour la dernière cellule de la liste, cette adresse doit valoir NULL.

Par exemple, la liste des trois entiers (12, 9, 25) doit être stockée en mémoire selon le schéma suivant :

Liste



Exercices (3/3)

- 1 Donnez la définition de la structure nécessaire à la manipulation de telles listes.
- 2 Écrivez une fonction permettant d'ajouter un entier en début de liste.
- 3 Écrivez une fonction permettant d'afficher à l'écran les entiers contenus dans une liste.
- 4 Écrivez une fonction permettant d'ajouter un entier en fin de liste.
- 5 Écrivez une fonction permettant de libérer l'espace mémoire occupé par une liste ; le résultat doit être une liste vide.

Pour chaque fonction, donnez-en un exemple d'appel.